Researches on Virtual Prototyping Presentation Model Based on Directed Acyclic Graph

Xiaoxi Zheng
Information school, WuYi University, Jiangmen, Guangdong, China 529020
Mailzxx@163.com

Guozheng Sun
CAD/CAE & Simulation Center, Wuhan University of Technology, Wuhan, Hubei, China, 430063

Shaomei Wang
CAD/CAE & Simulation Center, Wuhan University of Technology, Wuhan, Hubei, China, 430063

Abstract

The key technique of virtual prototyping is real-time a performance simulation for a product, such as virtual assembly simulation and mechanism movement simulation. However, the key to realize this simulation is yet the representation of a product model, and but, the traditional digital model of a product (such as relation model and hierarchical model) doesn't meet the demand. Therefore, this paper provides a scene graph model of a digital product based on DAG. The definition of the scene graph model is given and the features to represent a virtual product with DAG are analyzed。Finally, the principles of dynamically assembling and disassembling product with DAG and its simulating method are explained with an example.

Keywords: Virtual Prototyping, Virtual Assembly, Product Model, Directed Acyclic Graph.

1. Introduction

We can imagine a virtual prototyping (VP) system as the living theatre. To perform the play well, the objects of the characters of various roles, lighting and scene of the theatre would be required and then, showed them to audience (users) according to their time sequence and space position of these objects and their relationship each other in the play, which a story(digital product) will be interpreted with them. This ideal enlightens us that the various factor of "the evolution of a story" for building the digital mock-up (DMU) of VP should be thought as a whole, that is, with provision for the demands of product simulating process, the DMU for VP to build could freely be assembled and disassembled, and their operation could be facile and flexible. But because the first step of the main works of VP for simulating physical prototyping (PP) is that whole digital product is assembled with digital parts, and then the second, that the simulation of kinematics and dynamics is considered, the focus is always on study of simulation model for product assembly so that the requires of the simulating movement for mechanism to the product are overlooked. The result is that the simulation model of the product is less flexibility and adaptability, and it is difficult to following simulation of VP. So, this paper provides a new approach of representing dynamic digital product model based on Directed Acyclic Graph (DAG).

2. Related research

Research about assembly model of a product begins at the end of 70's. By studying many years, there are kinds of assembly models to be provided. They can be sorted into three type: relation model, hierarchy model and mix model.

2.1 Relation model [1]

Relation model is built with undirected graph. Its nodes denote parts of a product. Its edges express assembly relation, as Figure 1. The relation model can easily be created by computer and managed in the assembly process planning. Its disadvantages are that it cannot measure up to real structure of a product and human thinking habit due to representing information at same level. Furthermore, following assembly plan can be difficult if quantity of parts is greater.

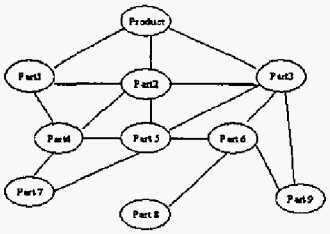

Figure 1 Relation model

2.2 Hierarchical model [2,3]

Hierarchical model is a tree structure with sub-assembly composed of some parts, as Figure 2. It just measures up to human thinking habit, and can represent design intent and product structure. By means of incorporated sub-assembly or some parts the numbers of elements at same level can be reduced and the complexity of assembly analysis can also be decreased. But there is a lack of obvious assembly relation. The rule to partition sub-assembly in the model isn't explicit.

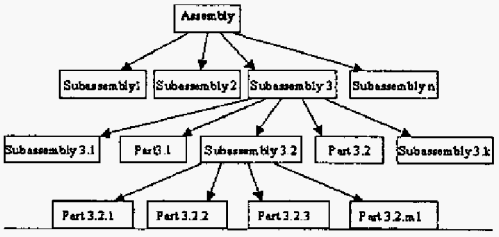

Figure 2 Hierarchical model

2.3 Mix model [4,5]

In practical commerce CAD systems, however, a mix model incorporated two models mentioned above into is always used. This model is a sub-assembly tree. Relation model is adopted between sibling nodes with same parents.

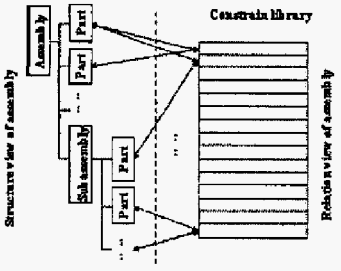

Figure 3 Mix model

In ref [5] an improved model based on mix model is provided in order to meet a demand to freely restructure the assembly model, which it is divided into two views: one is an assembly structure tree to only describe hierarchical structure of the assembly and another is a relation model to record all assembly relation between parts, as Figure 3.

As mentioned above, the problem of computer-aid assembly automation have mostly been solved with the three model and but, the active role of human in product design is ignored. Therefore, there are shortages as follows when these models are depicted as a virtual prototyping.

1) The role of intelligent decision-making of human in product assembly is lost. Virtually, physical product assembly is a process of inexactly intelligent reasoning of human, in which qualitative reasoning is important for decision-making, such as deciding movement trace of part assembly by seeing and feeling of sense organ. But this task is time consuming for planning by computer.

2) These models mentioned above describe only topological relation of product assembly and can't give its space position and direction between assemblies. But the process of physical product assembly has simultaneously included its space and topology relation.

3) Because their model can't unify the representations of space and topology relation between assemblies it is very difficult to operate real time and locally move it. The real time simulation of product assembly can also be supported.

4) Although hierarchical model and relation model can be combined together to cover their respective shorts their data structure is too complex due to creating independent constrain library (as Figure 3) so that search time increases. Furthermore, the form of the data structure is reticulated, and adding and deleting data dynamically isn't easy.

3. Directed acyclic graph——DAG

3.1 Definition of directed acyclic graph

Definition 1: A directed graph D is defined as an ordered pair $D=<V, E>$, where [6]

1) V is a finite and nonempty set in which the elements are called vertices;

2) E is a subset of Cartesian product $V\times V$, where $E\subseteq\{u \rightarrow v \mid u, v\in V, u\neq v\}$ in which the elements are called arcs or directed edges.

Definition 2: In the graph D if there exists an arbitrary ordering $v_1, v_2, \cdots\cdots, v_d$ of the vertices which is consistent with the graph D, that is $v_i \rightarrow v_j \in E$ implies $i < j$, then the graph D is called a directed acyclic graph(DAG). By the definition it is clear that the DAG doesn't contain any cycle, that is a path of the form $v \rightarrow \cdots\cdots \rightarrow v$, as Figure 4.

Definition 3: Let $u \rightarrow v$ $(u, v\in V)$ be a directed edge in DAG,. The vertex u is called a parent of v and v is a child of u. If there is $u \rightarrow v_i$ $(u, v_i\in V, i=1, 2, 3, \cdots\cdots, n)$ v_i are called siblings each other and if exists an ordering $v_1, v_2, \cdots\cdots, v_{i-1}, v_i$ of the vertices $v_1, v_2, \cdots\cdots, v_{i-1}$ are called the predecessors of v_i.

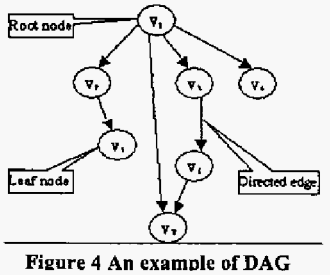

Figure 4 An example of DAG

3.2 Features of the DAG

1). Due to characteristic of the graph in the DAG, relation $u \rightarrow v \in E$ ($u \neq v$) can be set up with arbitrary two vertices u, $v \in V$. In other words, for arbitrary vertex $v_j \in V$, both a direct relation with $v_i \rightarrow v_j \in E$ and a indirect relation with a path of $v_i \rightarrow \cdots\cdots \rightarrow v_{j-1} \rightarrow v_j \in E$ can be set up, as Figure 5. Therefore, DAG can randomly set up the relations between objects.

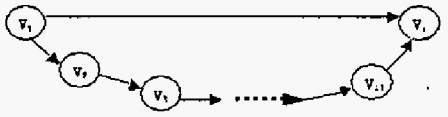

Figure 5 The directed and undirected relation of v_i and v_j

2). For arbitrary two vertices u, $v \in V$ ($u \neq v$) edge $u \rightarrow v$ is unequal to edge $v \rightarrow u$ with respect to directivity of DAG, where if there exists relation $u \rightarrow v$ the u is a parent of v and the v is a child. This property of the DAG shows to exist stable filiation between u and v. So when representing a body with this hierarchical relation the sequence and hypotaxis of its internal objects are clear. The representation to a body avails to the aggregation and separation of objects in it and to whole analysis and local operation, too.

3). For arbitrary vertices ordering $v_1, v_2, \cdots\cdots, v_d$, and $v_i \rightarrow v_j \in E$ we all have $i < j$ with respect to a-cycle of DAG. According to the property of DAG and any path $u \rightarrow \cdots\cdots \rightarrow v$(u, $\cdots\cdots$, $v \in V$) doesn't exist directed cycle $u \rightarrow \cdots\cdots \rightarrow v \rightarrow u$. Therefore, predecessor's relation can not turn back. With the feature the constrain relation between objects of a body could not exist to recursively depend on.

4). The characteristics of filiation and sibling of DAG make every vertex both to be connected and to be relatively independent. It is suitable for objects to be assembled level-by-level and disassembled. In addition, because of a-cycle of DAG the searching direction to vertices is definite and the efficiency of travel over the DAG is higher. Tracing to predecessors and seeking children from a vertex is very easy, flexible and speedy.

4. Virtual prototyping representation model based on DAG

Like physical prototyping does the aim of virtual prototyping must truly describe the assembly and movement process of 3D product. In other words, it is a process simulation with computer. This simulation should realistically imitate the process of assembly, disassembly and movement of 3D product. Therefore, the simulation has several characteristics as follow: real time, visualization and controllability. These characteristics determine that solving virtual prototyping isn't a problem of ordinary computer-aid assembly planning. To realize final aim of virtual prototyping, we should consider the representation of a virtual prototyping digit model as a whole. Thus, according to analysis above mentioned to DAG this paper provides a dynamical representing model of virtual prototyping based on DAG

4.1 Scene and entity

Definition 4: Scene is a "stage" in a virtual prototyping system on which shows all sorts of roles. As a role management space and visualization environment it includes entity objects of geometrical elements to be displayed, world coordinate system to position entities and lighting objects to light.

Definition 5: Entity is a graphic object to display, which consists of geometrical elements and their status in the scene. As a high level encapsulation for graphic object in the scene it can be used to represent a product, as a car, a motor, and to represent a part, as a piston, a linkage.

According to definition above mentioned, we can know that the scene is a stage to show a product and entity is a high level frame to represent a product. By managing the entities we can create, render and show a product while realize simulating aim of virtual prototyping. The entities are managed with DAG by the scene, as Figure 6.

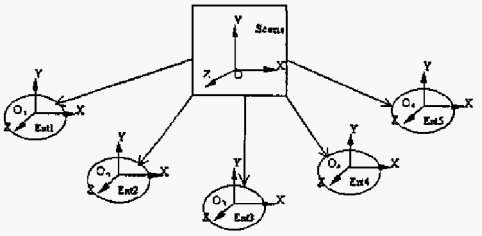

Figure 6 The model of entity management with GAG in the scene

The world coordinate system in scene is a parent node or root node. The entities are child nodes or leaf nodes when a entity is considered as a whole. This model has the advantages, as follow:

1) The function of dynamic adding and deleting entity can be realized by object list management.

2) Rendering operation to entity is easy and convenient by directly traveling entity object-list.

3) As an independent object the entity can represent both a complex product consisted of many parts and a single part, as Figure 7. Furthermore, because multi-entity can coexist in one scene it is easy to realize human-product interaction mechanism by picking up one entity with mouse.

4.2 Scene graph

The entity is a high level encapsulation. To detail the entity object in-depth we also adopt the DAG (called scene graph) to represent geometry of parts in the entity object.

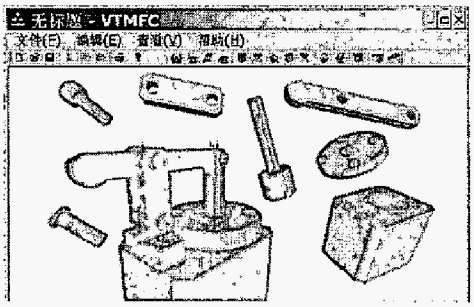

Figure 7 An example of a product and parts representing with entity

Definition 6: Scene graph is a data structure of graphic object and its status property represented with DAG. It is described as follow:

SG=(G, E, M), Where,

G is a geometrical model node,

E is a directed edge to represent hierarchical constrain relation from parent to child,

M is a local transform node.

Definition 7: In the scene graph the geometrical model node to simultaneously be directed with transform nodes from two paths is called nonlinear node, and, on the contrary, called linear node. The scene graph with nonlinear node is called a nonlinear scene graph, and, on the contrary, called linear scene graph.

The data model about geometrical model and constrain relation generated according to the definition is invoked by root node. Root node data pointer is saved into scene graph pointer variable in the entity data structure. In this way, a product model can be associated with the entity frame. When need to render scene we can pick up the data according to the scene graph pointer variable from it.

4.3 The principles of the scene graph data model based on DAG

We might as well investigate an example.

Figure 8 is the assembly example of a simplified product. The traditional assembly method is that the edge in graph is described as constrain relation and the part position in space is decided by only human-machine interaction with "mate", "co-axial", "offset". In this way, if once the part is assembled upon it is always fixed, and real time assembly and movement simulation couldn't be realized. However, the assembly procedure of physical product is in fact that the part in space is constantly moved to be by degrees close to assembly point and finally positioned on it. If the procedure is processes with computer it will be that the part model is constantly transformed to assembly point in virtual space until it is accurately positioned.

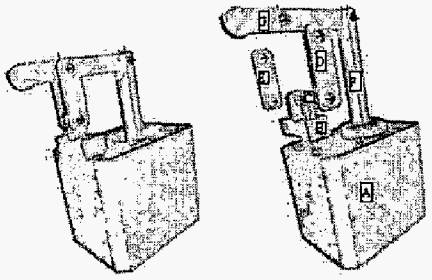

Figure 8 An example of a product assembling

Therefore, each part G to be assembled on will be associated with its transform matrix M in the world coordinate system. The movement status of the part G wholly depends on the value of M. In other words, If the value of M is given the status of the part G should be determinate, and, also, if the value of M is constantly changed the status of the part G could constantly be changed, where the movement status of the part G is M*G. Hence, this method to be used to simulate the physical assembly procedure quite accords with the ideal of physical assembly. On the other hand, product assembly is similar to put up building blocks. The parts are put up one layer by one layer to assemble into a product. The relatively stable position relation built among the parts couldn't be changed even though the position of a product assembly is changed in the world coordinate system. Furthermore, the movement of mechanism of a product is also a relative motion. It doesn't depend on the absolute position of a product in the world coordinate system. This tells us that a relative transform matrix among the parts should be taken as the M and isn't an absolute transform matrix relative to the world coordinate system. Through analysis above-mentioned the transform constrain relation in the Figure 8 example can be described a structure as Figure 9.

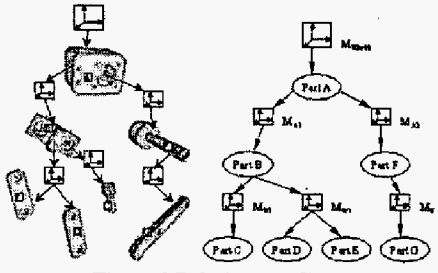

Figure 9 Relative coordinate transform between parts for a product

From Figure 9 we can see that the model coordinate of part A as a base can accord with the world coordinate. Part B under action of local transform M_{A1} relative to part A is positioned to its goal assembly pointer on part A by $M_{A1}*G_B$. Also, part C under action of local transform M_{B1} relative to part B is positioned to its goal assembly pointer on part B by $M_{B1}*G_C$, and so do other parts. Furthermore, when need for part C relative to part B to move we only changed the value of the transform M_{B1} and compute $M_{B1}*G_C$. In the same way, we can realize the movement of part B relative to part A. However, the change of relative transform matrix can't affect the status of assembly below it or dissociated from it. It is very easy to locally operate to parts and simulate relative movement of parts. This quite accords with thinking habit of human. That is just what we think. For this reason we can describe the assembly structure above-mentioned with DAG and realize dynamic scene graph model of a digit product as Figure 10.

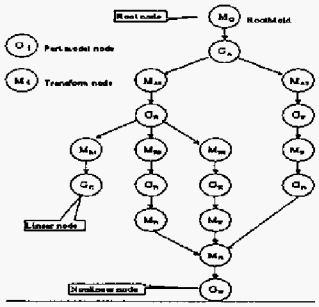

Figure 10 A product model representing with the scene graph

5. Conclusion

Through analysis above-mention we can observe that the scene graph based on DAG to represent a product model has some features as follows:

1) The scene graph model based on DAG describes geometric constrain relation as a node, which simplifies its data structure and saves time for processing data.

In the product model based on graph such as relation model, hierarchical model and mix model above-mentioned, traditional method is that a node is only described as a geometric model and an edge is only a geometric constrain relation. But generally, because the geometric constrains among parts imply both the topology structure and assembly position relation between parts one edge can't express two meaning. So it needs to build constrain library to express the position relation between parts as Figure 3, and then, this data structure is complicated, dynamic adding and deleting node is difficult. If use the scene graph based on DAG to represent a product model these problems could be solved.

2) Due to directing property between nodes model hierarchy is very clear and accords with thinking structure of human.

The scene graph is based on DAG. According to definition DAG has both properties of tree and graph, and so, it has advantages of both hierarchical model and relation model.

3) The capacity to describe complex product model with nonlinear node is strong.

The directing property of DAG improves the representation of linear structure about one parent and multi-child. But there is often a complex instance, that is, which one part is associated with several assemblies or parts below it. So this part needs to be described with nonlinear node in the scene graph and built a relation about multi-parent and one child. Especially, the presentation to close mechanism of a product is easy.

4) A linear node has a feature of independent movement and is suitable for relative rotating or translating simulation for assembly and part. A nonlinear node has a feature of associated movement and is suitable for movement simulation for mechanism of a product.

5) In the scene graph a node can be added into or deleted out and the product model with the scene graph is a dynamic structure, which parts can dynamically be assembled or disassembled. At the same time, the information of node in the scene graph is independency and inheritable character, which benefits to realize with OOP.

References

[1] Mantyla, M., A Modeling system for Top-Down Design of Assembled Products, *IBM J. Res. Develop.*, 34(5), 636-659, 1990

[2] David Neville Rocheleau and Kunwoo Lee, System for interactive assembly modeling, *Computer Aided Design*, 1987, 19(2): 65~72

[3] Lee K, Gossard D C. A hierarchical data structure for representing assemblies: Part 1 , *Computer-Aided Design*, 17(1), 15-19, 1985

[4] Tang Detong, A data model for product structure design, *J.CAD&CG*, 2000, 12(1), 11~16

[5] Zeng Li, Zhang Linsuan,, Xiao Tianyuan, Realization of a virtual assembly supported system, *J. System Simulation*, 14(9), 1149-1153, 2002

[6] Wang Zhaoduan, Graph theory (3ed), Beijing University of Technology Press, Beijing, 2001, p238